Prezime i ime učenika : *Stoiljković Mihajlo*

Odeljenje i grupa : *IV4 – grupa 3.*

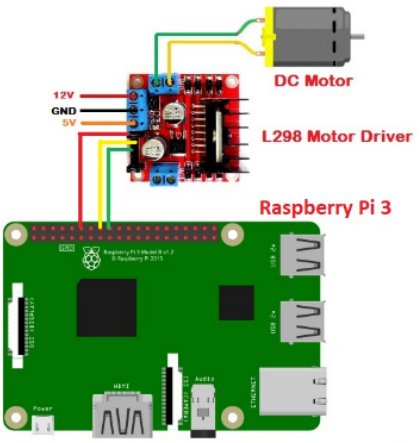
R. broj i naziv vežbe : *13.Управљање радон DC, серво и корачних мотора, контрола брзине и смера. Прављење робота са 2-4 точка/мотора, кретање, контрола кретања робота преко веб интерфејса*

IZVEŠTAJ:

DC motor (jednosmerni motor)

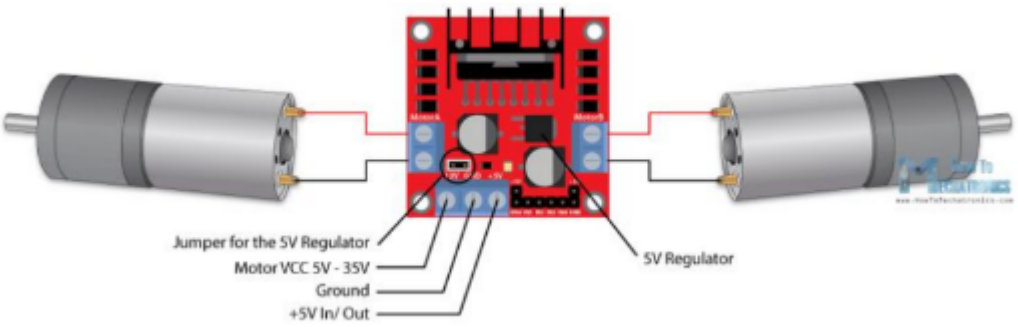
Jednosmerni motor je električni motor koji pretvara električnu energiju jednosmerne struje (DC) u mehaničku energiju, odnosno rotaciju osovine. Princip rada je da kada jednosmerna struja sa računara ili nekog jednosmernog izvora napajanja prođe kroz namotaje motora, stvara se magnetno polje. To magnetno polje reaguje sa stalnim magnetima ili drugim elektromagnetima u motoru. Zbog te interakcije nastaje sila koja okreće rotor (pokretni deo motora), i rotor se okreće i time proizvodi mehaničko kretanje.

Glavni delovi DC motora su:
• **Stator** – nepokretni deo koji stvara magnetno polje
• **Rotor** – deo koji se okreće
• **Komutator** – menja smer struje u namotajima
• **Četkice** – prenose struju sa napajanja na komutator



Ovi motori se često koriste u robotici, ventilatorima, hardverskim projektima sa mikrokontrolera ili mikroračunara, kontrola preko softvera i slično.

Drajver kod DC motora, kao i kod srodnih motora, predstavlja elektronsko kolo koje služi kao posrednik između mikrokontrolera i motora. Recimo, sam GPIO pin kod mikroračunara Raspberry Pi ima veoma malu struju (~mA), a sam DC motor zahteva stotine mA jačinu struje. Iz tog razloga koristimo drajver koji obezbeđuje dovoljnu struju za motor, štiti sam mikroračunar i omogućava kontrolu smera i brzine. Uz pomoć PWM signala se reguliše sama brzina jednosmernog motora. Za tačnu kontrolu položaja potreban je enkoder.



Servo motor

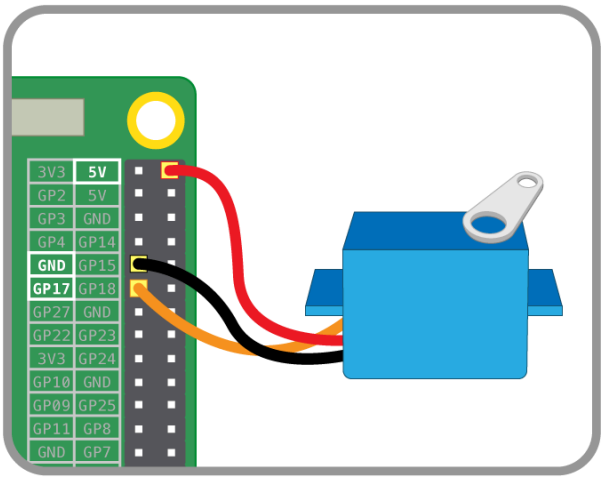
Servo motor je poseban tip električnog motora koji može vrlo precizno da kontroliše položaj, odnosno ugao, brzinu i kretanje. Najčešće se koristi kada treba da se okrene na tačno određen ugao, na primer **90° ili 180°**. Postoji više tipova servo motora koji se razlikuju po maksimalnom uglu rotacije (**90°, 180°, 360°**).

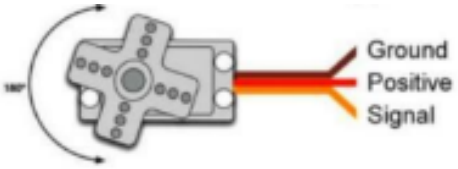
Servo motor u sebi ima nekoliko delova:
• **Mali DC motor**
• **Reduktor (zupčanici)** – smanjuju brzinu i povećavaju snagu
• **Potenciometar** – meri trenutni položaj osovine
• **Kontrolnu elektroniku**



Proces rada je sledeći: mikroračunar pošalje **PWM signal**, elektronika u servo motoru proverava na koji ugao treba da ide, zatim servo poredi željeni ugao i trenutni ugao. Na osnovu toga motor se okreće u odgovarajuću stranu dok ne dostigne zadati ugao.

Servo motori se često koriste u oblasti **robotike, automatizacije, modelarstva i raznih mehatroničkih sistema**.





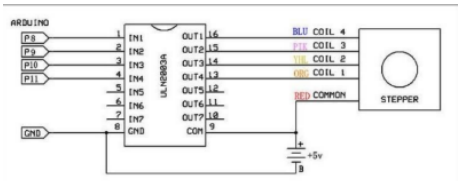
Koračni (Stepper) motor

Koračni motor je električni motor koji se ne okreće **kontinuirano**, već se okreće u malim preciznim koracima (stepovima). Zbog toga se može tačno kontrolisati koliko će se motor okrenuti, bez dodatnih senzora.

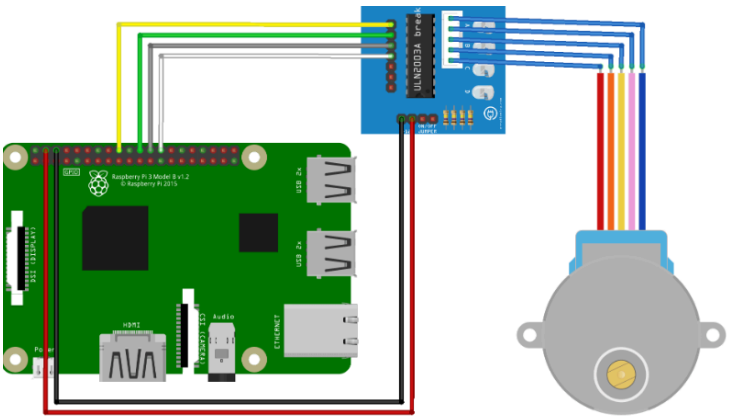
Unutar motora postoje **namotaji (elektromagneti)** koji se uključuju redom. Aktivira se prvi namotaj pa se rotor malo pomeri, zatim se aktivira sledeći i rotor napravi sledeći korak i tako redom.

Na primer, jedan korak iznosi **1.8°**, pa je za pun krug (**360°**) potrebno oko **200 koraka**.

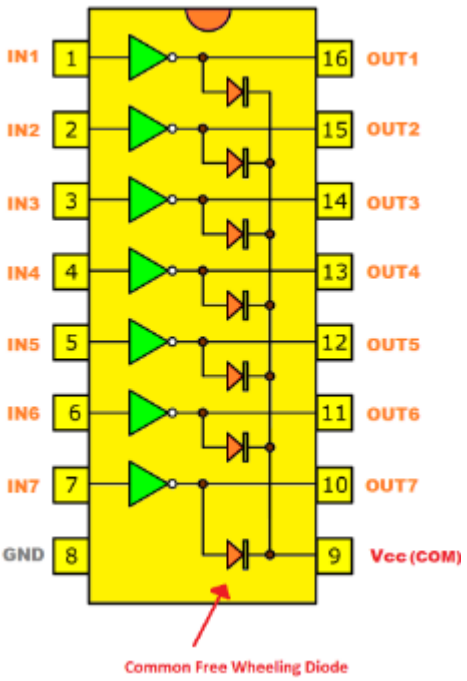
Glavni delovi koračnog motora su:
• **Stator** – sadrži elektromagnete
• **Rotor** – deo koji se okreće
• **Namotaji** – stvaraju magnetno polje



Koračni motori se često koriste kod **3D štampača, industrijske automatike, CNC mašina, printera i sličnih uređaja gde je potrebna precizna kontrola kretanja**.







Takođe i kod ovog stepper motora se koriste drajveri zbog napajanja i kontrole (**ULN2003**)

Vežba – projekat

U ovoj vežbi se realizuje kontrola jednosmernog (DC), koračnog i servo motora preko web aplikacije koristeći Python biblioteku Flask.

Библиотека Flask

Flask je Python web framework za razvoj web aplikacija i web servera. Omogućava da uz pomoć Pythona napravimo web interfejs za prikaz podataka i slično. Nudi minimalističku strukturu, omogućavajući nam da dodamo samo potrebna proširenja prema projektu.

Programeri ga često biraju zbog njegove jednostavne strukture i dobre podrške za integraciju sa bazama podataka. Flask takođe podržava razne dodatke (plugins) za rad sa bazama podataka, autentifikaciju korisnika, izradu REST API servisa i druge funkcionalnosti.

Flask funkcioniše tako što programer definiše rute (URL adrese) koje povezuju web zahteve sa funkcijama u Python kodu (rutiranje). Kada korisnik pristupi određenoj adresi u pregledaču, Flask pokreće odgovarajuću funkciju i vraća odgovor u obliku HTML stranice ili podataka. Ova biblioteka koristi templating engine koji omogućava povezivanje Python koda i HTML stranica. Na taj način se dinamički generiše sadržaj web stranice.

Struktura projekta:

RobotKontrola/

```
├── webapp.py
└── templates/
    └── index.html
```

Instalacija biblioteka - Linux

Pre početka razvoja sistema, potrebno je instalirati odgovarajuće biblioteke koje sadrže funkcije i ostale podatke neophodne za razvoj. Prvo ćemo osvežiti listu dostupnih paketa i verzija iz repozitorijuma.

```
mihajlo@DESKTOP-BAFHUP9:~$ sudo apt update
```

Nakon osvežavanja liste, sledi instalacija python3 flask biblioteke:

```
mihajlo@DESKTOP-BAFHUP9:~$ sudo apt install python3-flask
```

Ukoliko na Raspberry Pi nemamo biblioteku za rad sa GPIO pinovima, ulazima izlazima i konfiguracijom numeracije, instaliramo bibliteku:

```
mihajlo@DESKTOP-BAFHUP9:~$ pip3 install RPi.GPIO
```

Python web aplikacija (server + kontrola motora) - backend

U kodu ispod sledi python web aplikacija za kontrolu rada motora:

```python
from flask import Flask, render_template, request
import RPi.GPIO as GPIO
import time

app = Flask(__name__)
# ---------------- GPIO Setup ----------------
GPIO.setmode(GPIO.BCM)
# ---------------- DC MOTOR ----------------
IN1 = 17
IN2 = 27
ENA = 22
GPIO.setup(IN1, GPIO.OUT)
GPIO.setup(IN2, GPIO.OUT)
GPIO.setup(ENA, GPIO.OUT)
pwm_dc = GPIO.PWM(ENA, 1000)  # PWM na 1 kHz
pwm_dc.start(0)  # Start sa 0% duty cycle

# ---------------- SERVO ----------------
SERVO = 18
GPIO.setup(SERVO, GPIO.OUT)
servo_pwm = GPIO.PWM(SERVO, 50)  # Servo PWM 50 Hz
servo_pwm.start(7.5)  # Neutral position
# ---------------- STEPPER ----------------
step_pins = [5, 6, 13, 19]
for pin in step_pins:
    GPIO.setup(pin, GPIO.OUT)
# Redosled koraka za 28BYJ-48 stepper
step_seq = [
    [1,0,0,1],
```

```
    [1,0,0,0],
    [1,1,0,0],
    [0,1,0,0],
    [0,1,1,0],
    [0,0,1,0],
    [0,0,1,1],
    [0,0,0,1]
]
def step_motor(steps, direction):
    """Rotira stepper motor zadati broj koraka u datom smeru"""
    seq = step_seq if direction == 1 else step_seq[::-1]
    for _ in range(steps):
        for step in seq:
            for pin in range(4):
                GPIO.output(step_pins[pin], step[pin])
            time.sleep(0.001)
# ---------------- FLASK ROUTES ----------------
@app.route('/')
def index():
    return render_template('index.html')
@app.route('/dc', methods=['POST'])
def dc():
    speed = int(request.form['speed'])
    direction = request.form['direction']
    if direction == "forward":
        GPIO.output(IN1, 1)
        GPIO.output(IN2, 0)
    else:
        GPIO.output(IN1, 0)
        GPIO.output(IN2, 1)
    pwm_dc.ChangeDutyCycle(speed)
    return "DC motor updated"
@app.route('/servo', methods=['POST'])
def servo():
    angle = int(request.form['angle'])
    duty = 2 + (angle / 18)
    servo_pwm.ChangeDutyCycle(duty)
    time.sleep(0.5)
    servo_pwm.ChangeDutyCycle(0)
    return "Servo moved"
@app.route('/stepper', methods=['POST'])
def stepper():
    direction = int(request.form['direction'])
    step_motor(200, direction)  # jedan puni krug = 200 koraka
    return "Stepper moved"
# ---------------- RUN APP ----------------
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

U ovoj aplikaciji, na početku importujemo sve module i biblioteke neophodne za rad (Flask, render templejt, HTTP zahtevi, time, RPi.GPIO). Kao i svaka flask aplikacija predstavljena je objektom klase Flask tako što pozovemo konstuktor. Promenljiva **__name__** predstavlja ugrađenu promenljivu koja pokazuje naziv trenutnog modula. U sledecem delu, definišemo promenljive za pinove, numeraciju pinova kao BCM (procesor se gleda) kao i PWM signali motora. Nakon toga kreiramo listu pinova step motora i postavljamo kao izlazni. U sledećem delu definišemo matricu sa redosledom koraka za stepper motor i funkciju koja rotira stepper motor prema zadati broj koraka u datom smeru. Dalji deo koda aplikacije je fokusiran na flask rutiranje. Rutiranje zapravo predstavlja izbor funkcije

koja će biti pozvana da obradi pristup **HTTP** zahteva. Da bismo označili šta želimo da se rutira prema određenoj funkciji, potrebno je da u liniji iznad definicije funkcije navedemo odnosno dekorator za rutiranje koji ima oblik:

```
@app.route(...)
```

Na početku vidimo da imamo rutu **“/”** što znači da će ta funkcija index u ovom slučaju biti pozvana kada se navede adresa našeg veb sajta, bez dodatnih putanja (biće pozvana pri pokretanju aplikacije).

Sledeća funkcija **dc()** je namenjena za rutu gde se vrši kontrola DC motora I definisana je rutom **“/dc”** i metodom **HTTP POST** zahteva, što znači da se podaci šalju serveru. Unutar funkcije imamo konkretnu brzinu i direkciju motora, i ukoliko je direkcija definisana kao napred onda ispisujemo prema zadatom smeru, u suprotnom prati korake suprotno. Ova funkcija vraća string koji kaže da je DC motor ažuriran.

Naredna funkcija servo() je namenja za rutu gde se vrši kontrola servo motora i definisana je rutom **“/servo”** I metodom HTTP POST zahteva. Unutar funkcije imamo definisanje konkretno ugla I trajanja I na kraju se vraća string da je servo pomeren.

Naredna funkcija stepper() je namenja za rutu gde se vrši kontrola stepper motora i definisana je rutom **“/servo”** I metodom HTTP POST zahteva. Unutar funkcije imamo definisanje direkcije pokreta I vraćanje stringa da je stepper motor pomeren.

Na kraju za pokretanje same aplikacije proveravamo da li je ugrađena promenljiva ima vrednost **__main__** (da li je pokrenut direktno)I ukoliko ima, startujemo aplikaciju sa datim hostom I portom. Za dateljniji razvoj I debugging, može se postaviti parametar **debug = True** koji će javiti grešku na veb interfejsu ukoliko postoji.

Html web interfejs – frontend

```
<!DOCTYPE html>
<html>
<head>
    <title>Motor Control</title>
</head>
<body>
    <h2>DC Motor</h2>
    <form action="/dc" method="post">
        Speed (0-100): <input type="number" name="speed"><br>
        Direction:
        <select name="direction">
            <option value="forward">Forward</option>
            <option value="backward">Backward</option>
        </select>
        <input type="submit">
    </form>

    <h2>Servo Motor</h2>
    <form action="/servo" method="post">
        Angle (0-180): <input type="number" name="angle">
        <input type="submit">
    </form>

    <h2>Stepper Motor</h2>
    <form action="/stepper" method="post">
```

```
        Direction:
        <select name="direction">
            <option value="1">Forward</option>
            <option value="-1">Backward</option>
        </select>
        <input type="submit">
    </form>
</body>
</html>
```

Ovaj web interfejs se sastoji od **tri forme**, svaka sa svojom funkcionalnošću za kontrolu motora. Svaka forma koristi **akciju (action)** koja je povezana sa odgovarajućom Flask rutom i **metodu POST** za slanje podataka.

🎬 **DC Motor**:

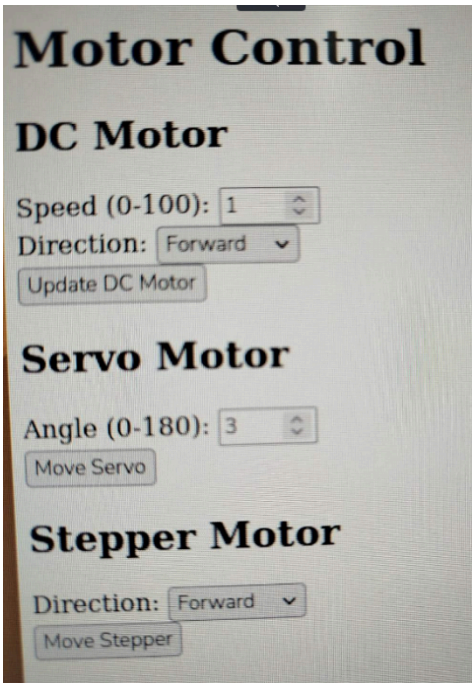
- Forma šalje **brzinu i smer** na Flask rutu `/dc` metodom POST.
- Korisnik može uneti brzinu 0–100 i izabrati smer (Forward / Backward).

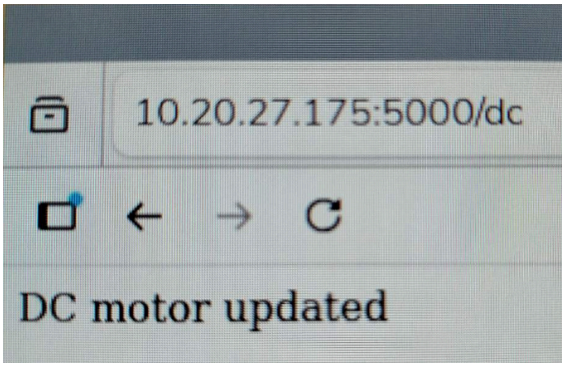
🎬 **Servo Motor**:

- Forma šalje **ugao** (0–180°) na rutu `/servo`.
- Servo se okrene na zadati ugao preko PWM signala.

🎬 **Stepper Motor**:

- Forma šalje **smer** (`1` = napred, `-1` = nazad) na rutu `/stepper`.
- Funkcija `step_motor()` rotira stepper motor zadati broj koraka.









Prikaz **HTTP POST** zahteva/metoda:

```
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.20.27.175:5000
Press CTRL+C to quit
10.20.27.175 - - [09/Mar/2026 17:23:30] "GET / HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:23:42] "POST /servo HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:23:54] "POST /servo HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:24:02] "POST /servo HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:24:13] "POST /servo HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:24:17] "POST /servo HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:24:22] "POST /servo HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:24:24] "POST /servo HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:24:27] "POST /servo HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:24:31] "POST /servo HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:24:52] "POST /servo HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:27:24] "POST /stepper HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:28:10] "POST /stepper HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:28:30] "POST /servo HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:29:09] "POST /servo HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:29:34] "POST /servo HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:29:38] "POST /servo HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:30:38] "POST /stepper HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:30:46] "POST /stepper HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:30:51] "POST /stepper HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:33:47] "POST /stepper HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:33:53] "POST /stepper HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:34:02] "POST /stepper HTTP/1.1" 200 -
```

Pokretanje web aplikacije iz terminala:

```
^Craspberrypi@raspberry2:~/Desktop/robot projekat $ python3 app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
 Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.20.27.175:5000
Press CTRL+C to quit
10.20.27.175 - - [09/Mar/2026 17:11:27] "GET / HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:11:33] "POST /dc HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:11:38] "POST /servo HTTP/1.1" 200 -
10.20.27.175 - - [09/Mar/2026 17:11:44] "POST /stepper HTTP/1.1" 200 -
```

